Computer Vision with PyTorch

From Pixels to Predictions: Building Image Classifiers

1

Part 1: Introduction to Computer Vision

2

What is Computer Vision?

- **Computer Vision (CV)**: Teaching computers to "see" and understand images
- Goal: Extract meaningful information from visual data
- **Key Tasks**:
- • Image Classification: What is in this image?
- • Object Detection: Where are the objects?
- • Semantic Segmentation: Pixel-level classification
- CV powers: self-driving cars, face recognition, medical imaging, and more!

3

CV Applications

- **Healthcare**: X-ray analysis, tumor detection, pathology slides
- **Autonomous Vehicles**: Lane detection, pedestrian recognition
- **Face Recognition**: Unlocking phones, security systems
- **Agriculture**: Crop disease detection, yield prediction
- **Manufacturing**: Quality control, defect detection
- The market is growing rapidly — CV skills are in high demand!

4

Part 2: Images as Data

5

How Computers See Images

- Humans see colors, shapes, and objects
- Computers see **numbers** — a grid of pixel values
- **Grayscale**: Single channel, values 0-255 (black to white)
- **Color (RGB)**: Three channels — Red, Green, Blue
- Each pixel: [R, G, B] values from 0-255

🚀 **Interactive Demo:** image_tensor_demo.html

6

Image Tensors in PyTorch

- Images are represented as **tensors** (multi-dimensional arrays)
- **Shape convention**: (Batch, Channels, Height, Width) or BCHW

PYTHON

```
import torch
from PIL import Image
import torchvision.transforms as T

# Load an image
img = Image.open('cat.jpg')  # Shape: (H, W, C)

# Convert to tensor
transform = T.ToTensor()  # Converts to (C, H, W), scales to [0
img_tensor = transform(img)

print(img_tensor.shape)  # torch.Size([3, 224, 224])
```

7

Why Normalize Images?

- Raw pixel values: 0-255 (integers)
- Neural networks prefer: 0-1 or standardized values
- **Standard ImageNet normalization**:

  PYTHON

  ```python
  transform = T.Compose([
      T.ToTensor(),  # [0, 255] → [0, 1]
      T.Normalize(
          mean=[0.485, 0.456, 0.406],  # ImageNet means
          std=[0.229, 0.224, 0.225]    # ImageNet stds
      )
  ])
  ```

- Normalization helps training converge faster and more stably

8

Part 3: Convolutions — The Core Operation

9

Why Not Fully Connected Layers?

- A 224×224 RGB image = 150,528 input features
- FC layer to 1000 neurons = **150 million parameters!** 😱
- Problems:
- • Too many parameters → overfitting
- • Ignores spatial structure of images
- • No position invariance
- Solution: **Convolutional layers** 🎯

10

What is a Convolution?

- A **filter/kernel** slides over the image
- At each position: element-wise multiply, then sum
- Produces a **feature map** highlighting patterns
- **Key insight**: The same filter is applied everywhere

🚀 **Interactive Demo:** convolution_demo.html

11

Convolution Math

- Input: $X \in \mathbb{R}^{H \times W}$, Kernel: $K \in \mathbb{R}^{k \times k}$
- Output at position (i, j):
- $$Y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m,j+n} \cdot K_{m,n}$$
- **Learned kernels** detect features: edges, textures, shapes
- Early layers: simple features (edges, gradients)
- Later layers: complex features (eyes, wheels, faces)

12

Conv2d in PyTorch

PYTHON

```python
import torch.nn as nn

# Convolution layer
conv = nn.Conv2d(
    in_channels=3,      # RGB input
    out_channels=16,    # 16 output feature maps
    kernel_size=3,      # 3×3 filter
    stride=1,           # Move filter by 1 pixel
    padding=1           # Add 1 pixel border to preserve size
)

# Input: (batch, 3, 32, 32)
# Output: (batch, 16, 32, 32)
```

- **Key parameters**: kernel_size, stride, padding

13

Output Size Formula

- Given input size W, kernel size K, padding P, stride S:
- $$W_{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$
- **Example**: Input 32×32, kernel 3×3, padding 1, stride 1:
- $$W_{out} = \left\lfloor \frac{32 - 3 + 2}{1} \right\rfloor + 1 = 32$$
- Same padding: $P = \lfloor K/2 \rfloor$ keeps size the same

14

Part 4: Pooling & More Layers

15

Pooling Layers

- **Purpose**: Reduce spatial dimensions (downsample)
- **Max Pooling**: Take maximum value in each region
- **Average Pooling**: Take average value in each region
- Benefits:
- • Reduces computation
- • Provides translation invariance
- • Reduces overfitting

🚀 **Interactive Demo:** pooling_demo.html

16

Pooling in PyTorch

PYTHON

```python
# Max pooling: 2×2 window, stride 2
pool = nn.MaxPool2d(kernel_size=2, stride=2)

# Input: (batch, 16, 32, 32)
# Output: (batch, 16, 16, 16)  — halves each dimension

# Adaptive pooling: output fixed size regardless of input
adaptive_pool = nn.AdaptiveAvgPool2d((1, 1))  # Global average
# Output: (batch, channels, 1, 1)
```

17

Activation & BatchNorm

- **ReLU**: Standard activation, $f(x) = \max(0, x)$
- **Batch Normalization**: Normalizes layer outputs

PYTHON

```
# Common pattern: Conv → BatchNorm → ReLU
nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True)
)
```

- BatchNorm benefits: faster training, higher learning rates, regularization

18

Part 5: Building a CNN

19

CNN Architecture Overview

- **Feature Extractor**: Conv layers learn representations
- **Classifier**: FC layers make predictions
- Typical pattern: [Conv → BN → ReLU → Pool] × N → FC

🚀 **Interactive Demo:** cnn_architecture_demo.html

20

Simple CNN for CIFAR-10

PYTHON

```python
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Feature extractor
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),  # 32→16

            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),  # 16→8

            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),  # 8→4
        )
        # Classifier
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        x = self.features(x)
```

21

```
        x = self.classifier(x)
        return x
```

Loading Image Datasets

PYTHON

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Define transforms
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

# Load CIFAR-10
train_dataset = datasets.CIFAR10(
    root='./data', train=True, download=True, transform=transfo
)
test_dataset = datasets.CIFAR10(
    root='./data', train=False, download=True, transform=transf
)

# Create data loaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=F
```

Training the CNN

PYTHON

```python
model = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f'Epoch {epoch+1}, Loss: {loss.item():.4f}')
```

23

Part 6: Data Augmentation

24

Why Data Augmentation?

- More data → better generalization
- **Data augmentation**: Create variations of existing images
- Helps models learn invariances:
- • Position: Random crops
- • Rotation: Random rotations
- • Lighting: Color jitter
- **Free extra data** without collecting new images!

25

Common Augmentations

PYTHON

```python
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.RandomCrop(32, padding=4),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

# For testing: NO augmentation, just normalize
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

Part 7: Transfer Learning

27

What is Transfer Learning?

- **Key insight**: Features learned on one task transfer to others
- A network trained on ImageNet (1M images, 1000 classes) learns:
- • Early layers: edges, textures (universal)
- • Later layers: eyes, wheels (task-specific)
- **Strategy**: Use pre-trained features, fine-tune for your task
- Benefits: Less data needed, faster training, better results!

28

Using Pre-trained Models

PYTHON

```
from torchvision import models

# Load pre-trained ResNet18
model = models.resnet18(pretrained=True)

# Freeze all layers (don't update during training)
for param in model.parameters():
    param.requires_grad = False

# Replace final layer for our task (e.g., 10 classes)
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 10)

# Only the new fc layer will be trained
model = model.to(device)
```

29

Fine-Tuning Strategies

- **Feature Extraction**: Freeze all, train only new layers
- • Fast, works with small datasets
- **Fine-Tuning**: Unfreeze some/all layers, train with low LR
- • Better performance, needs more data

PYTHON

```
# Unfreeze last few layers for fine-tuning
for name, param in model.named_parameters():
    if 'layer4' in name or 'fc' in name:
        param.requires_grad = True

# Use smaller learning rate for fine-tuning
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

30

Part 8: Famous CNN Architectures

31

Historical CNN Milestones

- **LeNet-5** (1998): First practical CNN, handwritten digits
- **AlexNet** (2012): Won ImageNet, sparked deep learning revolution
- **VGGNet** (2014): Very deep (16-19 layers), simple 3×3 convs
- **ResNet** (2015): Skip connections, 152+ layers possible
- **EfficientNet** (2019): Optimal scaling of depth/width/resolution

32

Available in torchvision

PYTHON

```
from torchvision import models

# Classic architectures
resnet = models.resnet50(pretrained=True)
vgg = models.vgg16(pretrained=True)

# Modern architectures
efficientnet = models.efficientnet_b0(pretrained=True)
convnext = models.convnext_tiny(pretrained=True)

# Vision Transformers (ViT)
vit = models.vit_b_16(pretrained=True)
```

- Choose based on: accuracy vs. speed vs. model size tradeoffs

33

Part 9: Beyond Classification

34

The CV Landscape

- Classification is just the beginning! The CV field is evolving:
- **Detection & Segmentation**: Find and outline objects
- **Video Understanding**: Temporal reasoning across frames
- **VLMs**: Vision + language for open-ended understanding
- **World Models**: Predict the future, simulate physics
- **VLA & Physical AI**: Robots that see, understand, and act!

🚀 **Interactive Demo:** cv_tasks_demo.html

35

Object Detection

- **Goal**: Locate objects with bounding boxes + classify them
- Output: (x, y, width, height, class, confidence) for each object
- **Key Models**:
- • **YOLO** (You Only Look Once): Real-time, single-stage
- • **Faster R-CNN**: Two-stage with region proposals
- • **DETR**: Transformer-based, end-to-end
- Applications: Self-driving cars, surveillance, retail analytics

36

Semantic & Instance Segmentation

- **Semantic Segmentation**: Every pixel gets a class label
- • All cats = same color, all dogs = same color
- **Instance Segmentation**: Distinguish individual objects
- • Cat 1 ≠ Cat 2, each gets unique ID
- **Key Models**: U-Net, Mask R-CNN, DeepLab, SAM
- **SAM** (Segment Anything): Zero-shot segmentation — works on anything!

37

Video Understanding

- **Beyond single images**: Videos add temporal dimension
- **Action Recognition**: What is happening over time?
- **Key Models**:
- • **VideoMAE**: Masked autoencoder for video
- • **TimeSformer**: Divided space-time attention
- • **Gemini**: Native multimodal, long video understanding
- Applications: Sports analytics, content moderation, surveillance

38

Vision-Language Models (VLMs)

- **See and Talk**: Combine vision encoders with LLMs
- Open-ended visual QA, image description, visual reasoning
- **Key Models**:
- • **GPT**: OpenAI's multimodal model
- • **Gemini**: Google's native multimodal
- • **LLaVA**: Open-source VLM
- The foundation for multimodal AI assistants!

39

World Models & Physical AI

- **World Models**: Learn to simulate environments
- • Predict how actions affect the world
- • Models: Sora, Genie, NVIDIA Cosmos
- **Physical AI**: The convergence of CV + NLP + Robotics
- • Humanoid robots: Tesla Optimus, Figure, 1X NEO
- • Home robots, manufacturing, healthcare
- The future: AI that physically interacts with our world 🚀

40

Vision-Language-Action (VLA) Models

- **The Bridge**: From understanding to action
- VLA = See 👁️ + Understand 💬 + Act 💪
- **Example**: 'Pick up the red cup' → robot executes!
- **Key Models**:
- • **RT**: Google's Robotics Transformer
- • **PaLM-E**: 562B parameter embodied LLM
- • **OpenVLA**: Open-source, fine-tuned from LLaVA
- • π_0: Physical Intelligence's VLA

41

All Interactive Demos

- 📊 **Visualization Tools:**

  🚀 **Interactive Demo:** image_tensor_demo.html

  🚀 **Interactive Demo:** convolution_demo.html

  🚀 **Interactive Demo:** pooling_demo.html

  🚀 **Interactive Demo:** cnn_architecture_demo.html

- 🔮 **Beyond Classification:**

  🚀 **Interactive Demo:** cv_tasks_demo.html

- Click any demo above to practice interactively.

42

Lecture Summary

- **Images as Tensors**: (B, C, H, W) format, normalize for training
- **Convolutions**: Slide filters to detect features, shared weights
- **CNN Architecture**: Conv → BN → ReLU → Pool → FC
- **Data Augmentation**: Free extra training data via transformations
- **Transfer Learning**: Leverage pre-trained models for better results
- **Beyond Classification**: Detection → Video → VLMs → World Models → VLA → Physical AI

43

Supplementary Resources

🚀 **Interactive Demo:** handout.html

- 📓 **Hands-on Tutorial**: Check the Jupyter notebook in `notebooks/` (in Google Drive lecture folder)
- Review handout for offline study.

44